

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ
УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТОТЕХНИЧЕСКИХ СИСТЕМ И
ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА

по дисциплине

«ИНТЕРНЕТ-ТЕХНОЛОГИИ»

на тему:

«Разработка веб-приложения для организации мероприятий»

Выполнил:

Пензев Константин Сергеевич

Студент 4 курса группы ПИН-б-з-22-1

Направления подготовки

09.03.03 Прикладная информатика

заочной формы обучения

(подпись)

Руководитель работы:

(ФИО, должность, кафедра)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г.

УТВЕРЖДАЮ

Директор департамента цифровых,
робототехнических систем

и электроники

Азаров И.В.

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Направление подготовки 09.03.03 Прикладная информатика
Направленность (профиль) Прикладная информатика в экономике

ЗАДАНИЕ

на курсовую работу

студента Пензева Константина Сергеевича

(фамилия, имя, отчество)

по дисциплине Интернет-технологии

1. Тема работы «Разработка веб-приложения для организации мероприятий»

2. Цель: разработать полноценное full-stack веб-приложение для организации, поиска и бронирования мероприятий, включающее клиентскую и серверную части, систему аутентификации с ролевой моделью, каталог событий с фильтрацией, выбор билетов, оформление бронирования и административную панель.

3. Технологический стек: Next.js 14 (App Router), TypeScript, React 18, Prisma ORM, Zustand, Tailwind CSS, Docker.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части: анализ предметной области организации мероприятий, обзор существующих решений, обоснование выбора технологического стека, принципы full-stack разработки и App Router.

б) по практической части: проектирование схемы БД (9 моделей: User, Category, Event, Ticket, Booking, BookingItem, Review, Favorite, EventImage), реализация REST API на Next.js Route Handlers, разработка клиентского интерфейса на React и Tailwind CSS, реализация корзины бронирования на Zustand, аутентификация на JWT (jose + bcryptjs) с middleware, ролевая модель (USER, ORGANIZER, ADMIN), контейнеризация с Docker и развёртывание на VPS.

5. Исходные данные: техническое задание на разработку веб-приложения для организации мероприятий «MyEventsPro», документация по Next.js 14, Prisma, React, Tailwind CSS, Docker.

6. Список рекомендуемой литературы: официальная документация Next.js, Prisma, React, TypeScript, Tailwind CSS, Docker.

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % -	“	”	20	г.
50 % -	“	”	20	г.
75 % -	“	”	20	г.
100 % -	“	”	20	г.

8. Срок защиты студентом курсовой работы “ ” 20_ г.

Дата выдачи задания “ ” 20_ г.

Руководитель курсовой работы

Старший преподаватель департамента цифровых, робототехнических систем и электроники института перспективной инженерии Щеголев А.А.

(ученая степень, звание)(личная подпись)(инициалы, фамилия)

Задание принял к исполнению студент заочной формы обучения 4 курса

ПИН-б-3-22-1 группы _____
(личная подпись) (инициалы, фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	9
1.1 Постановка задачи.....	9
1.2 Обзор существующих решений.....	10
1.3 Обоснование выбора технологии разработки	11
2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ.....	13
2.1 Функциональные требования.....	13
2.2 Нефункциональные требования	14
3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	16
3.1 Архитектура системы	16
3.2 Проектирование базы данных (Prisma ORM).....	19
3.3 Проектирование REST API	21
3.4 Проектирование пользовательского интерфейса.....	23
4 РЕАЛИЗАЦИЯ	24
4.1 Технологический стек (Next.js full-stack)	24
4.2 Реализация каталога мероприятий и фильтрации	25
4.3 Реализация корзины бронирования на Zustand.....	26
4.4 Реализация оформления бронирования	27
4.5 Аутентификация и ролевая модель (JWT + middleware)	29
4.6 Контейнеризация и развёртывание	31
5 ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА.....	33
5.1 Страница авторизации	33
5.2 Главная страница.....	34
5.3 Каталог мероприятий.....	35

5.4 Форма создания мероприятия.....	36
6 ТЕСТИРОВАНИЕ	37
6.1 Методика тестирования.....	37
6.2 Результаты функционального тестирования.....	37
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	43

ВВЕДЕНИЕ

В современном мире организация мероприятий — конференций, концертов, выставок, мастер-классов, спортивных событий — является важной частью культурной, образовательной и бизнес-среды. Эффективная организация и посещение мероприятий требует удобных инструментов для публикации информации о событиях, поиска интересующих программ, выбора и покупки билетов, а также управления бронированиями. Традиционные подходы, основанные на ручной продаже билетов или разрозненных системах, обладают рядом недостатков: сложностью актуализации информации, ограниченными возможностями поиска и фильтрации, отсутствием единого профиля пользователя и истории посещений.

Современные веб-приложения для организации мероприятий лишены указанных недостатков. Они обеспечивают централизованное хранение информации о событиях, удобный каталог с фильтрацией и сортировкой, онлайн-выбор билетов и оформление бронирования, личный кабинет пользователя с историей заказов, а также административные инструменты для управления мероприятиями. В связи с этим разработка современного full-stack веб-приложения для организации мероприятий является актуальной задачей.

Объектом исследования является процесс full-stack разработки веб-приложений с использованием современных фреймворков, обеспечивающих совмещение серверной и клиентской логики в едином проекте.

Предметом исследования выступают методы и средства создания веб-приложения для организации мероприятий, включая проектирование базы данных с помощью Prisma ORM, реализацию REST API на базе Next.js App Router, организацию ролевой модели доступа (пользователь, организатор, администратор), аутентификацию на основе JWT с

middleware-защитой маршрутов, управление состоянием корзины бронирования на Zustand и контейнеризацию приложения с использованием Docker.

Целью курсовой работы является разработка full-stack веб-приложения для организации, поиска и бронирования мероприятий «MyEventsPro», обеспечивающего публикацию и поиск мероприятий по категориям, выбор типов билетов, оформление бронирования с генерацией кода подтверждения, личный кабинет пользователя с историей бронирований, возможность создания мероприятий организаторами и административное управление контентом.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и обзор существующих решений для организации мероприятий;
- сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
- спроектировать архитектуру full-stack приложения на базе Next.js 14 с App Router;
- спроектировать базу данных с использованием Prisma ORM (9 моделей);
- реализовать REST API на Next.js Route Handlers для работы с мероприятиями, бронированиями и пользователями;
- реализовать клиентский интерфейс на React 18 и Tailwind CSS;
- реализовать каталог мероприятий с фильтрацией по категориям и параметрам;
- реализовать выбор билетов и корзину бронирования на Zustand с сохранением в localStorage;
- реализовать оформление бронирования с генерацией уникального кода подтверждения;

- организовать аутентификацию пользователей на основе JWT (jose + bcryptjs) с middleware-защитой;
- реализовать ролевую модель (USER, ORGANIZER, ADMIN) с разграничением доступа;
- реализовать форму создания мероприятий для организаторов и административную панель;
- выполнить контейнеризацию приложения с использованием Docker и Docker Compose;
- развернуть приложение на VPS-сервере с использованием Nginx и SSL-сертификатов Let's Encrypt;
- провести функциональное тестирование основных сценариев работы приложения.

Практическая значимость работы заключается в том, что разработанное приложение может быть использовано в качестве готовой платформы для организации и продажи билетов на мероприятия, а также в качестве учебного примера построения современных full-stack веб-приложений с ролевой моделью доступа.

Структура работы. Курсовая работа состоит из введения, пяти глав, заключения и списка использованных источников. В первой главе проводится анализ предметной области. Во второй главе формулируется техническое задание. В третьей главе рассматриваются вопросы проектирования системы. Четвёртая глава посвящена реализации. В пятой главе описываются результаты тестирования. Объём работы составляет 30–40 страниц.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Постановка задачи

Организация мероприятий — это комплексный процесс, включающий публикацию информации о событии, продажу билетов, учёт бронирований и управление контентом. Для организатора важно иметь удобный инструмент создания и публикации карточки мероприятия с описанием, датами, местом проведения, типами билетов и ценами. Для посетителя (пользователя) важно иметь возможность искать мероприятия по категориям и параметрам, просматривать детальную информацию, выбирать подходящие типы билетов, оформлять бронирование и отслеживать свои заказы в личном кабинете.

Традиционные подходы к организации продажи билетов (кассовая продажа, ручной учёт, электронные таблицы) обладают рядом существенных недостатков: ограниченной доступностью (только в рабочие часы кассы), сложностью актуализации информации, отсутствием удобного поиска и фильтрации, высокой вероятностью ошибок при ручном учёте. Современные веб-приложения устраняют эти недостатки, обеспечивая круглосуточный доступ, автоматизацию процесса бронирования и централизованное управление контентом.

В рамках данной курсовой работы разрабатывается full-stack веб-приложение для организации мероприятий «MyEventsPro», которое должно сочетать удобство использования с современными подходами к разработке программного обеспечения. Приложение должно предоставлять три уровня доступа: пользователь (USER) — поиск и бронирование мероприятий; организатор (ORGANIZER) — создание и управление своими мероприятиями; администратор (ADMIN) — полное управление всеми мероприятиями и пользователями.

Разрабатываемое приложение должно быть построено на базе фреймворка Next.js 14 с использованием App Router, что позволяет совместить серверную и клиентскую логику в одном приложении. Такая

full-stack архитектура обеспечивает ряд преимуществ: единый код для frontend и backend, встроенную поддержку серверного рендеринга (SSR), типобезопасность на уровне всего стека благодаря TypeScript и упрощённое развёртывание.

Дополнительно приложение должно поддерживать различные типы мероприятий (онлайн и офлайн), категории (концерты, конференции, выставки, мастер-классы, спорт, другие), несколько типов билетов для одного мероприятия с разными ценами, систему отзывов и оценок, а также избранные мероприятия.

1.2 Обзор существующих решений

На рынке платформ для организации и продажи билетов на мероприятия представлен ряд решений, различающихся по функциональности, охвату и стоимости. Среди наиболее известных можно выделить следующие.

Kassir.ru — крупный российский сервис продажи билетов на концерты, спектакли, спортивные и другие мероприятия. Предоставляет удобный каталог с фильтрацией, выбор мест на схеме зала, онлайн-оплату и электронные билеты. Отличается широким охватом, однако является закрытой коммерческой платформой, не доступной для самостоятельного развёртывания.

Timepad — платформа для организации мероприятий и продажи билетов, предоставляющая инструменты для создания лендингов событий, продажи билетов, регистрации участников и аналитики. Ориентирована на организаторов, но является коммерческим SaaS-решением.

Radario — сервис для продажи электронных билетов и управления мероприятиями. Предоставляет инструменты для создания событий, продажи билетов, контроля доступа и аналитики.

Eventbrite — международная платформа для организации мероприятий, позволяющая создавать страницы событий, продавать билеты (в том числе бесплатные), управлять регистрацией и проводить

маркетинговые кампании. Является одним из лидеров мирового рынка, но недоступна для самостоятельного развёртывания.

Анализ существующих решений позволяет сформулировать следующие выводы. Во-первых, большинство популярных платформ являются облачными SaaS-сервисами и не предоставляют возможности самостоятельного развёртывания. Во-вторых, многие из них имеют комиссию с продажи билетов и платные подписки. В-третьих, для небольших организаторов и образовательных учреждений потребность в лёгкой, расширяемой и доступной для самостоятельного развёртывания платформе остаётся актуальной. Указанные факторы обуславливают целесообразность разработки собственного full-stack веб-приложения на современном технологическом стеке.

1.3 Обоснование выбора технологии разработки

Для реализации веб-приложения был выбран современный full-stack технологический стек, обеспечивающий высокую производительность, удобство разработки и возможность дальнейшего масштабирования.

В качестве основного фреймворка выбран Next.js версии 14.2.5 — один из наиболее популярных React-фреймворков, обеспечивающий поддержку серверного рендеринга (SSR), статической генерации (SSG) и клиентского рендеринга (CSR). Ключевым преимуществом Next.js 14 является App Router — новая система маршрутизации, основанная на файловой системе, которая позволяет совмещать серверные и клиентские компоненты в одном проекте. Использование App Router устраняет необходимость в отдельном backend-сервере: серверная логика реализуется в Route Handlers (файлы route.ts), а страницы — в Server Components.

В качестве языка программирования выбран TypeScript — статически типизированное надмножество JavaScript. Использование TypeScript обеспечивает выявление ошибок на этапе компиляции, улучшает читаемость кода и упрощает его поддержку. TypeScript

применяется как на клиенте, так и на сервере, что обеспечивает единообразие разработки.

Для работы с базой данных выбран Prisma ORM — современный объектно-реляционный маппер, предоставляющий типобезопасный доступ к данным. Prisma генерирует TypeScript-типы на основе схемы базы данных, что обеспечивает автодополнение и проверку типов в запросах. В качестве СУБД используется SQLite — лёгкая встроенная файловая база данных, не требующая отдельного сервера.

Для управления клиентским состоянием (корзина бронирования) выбран Zustand — лёгкая и быстрая библиотека управления состоянием для React. Zustand отличается простотой API, отсутствием boilerplate-кода и поддержкой middleware persist для сохранения состояния в localStorage.

Для стилизации интерфейса применяется Tailwind CSS версии 3.4 — utility-first CSS-фреймворк, позволяющий создавать дизайн непосредственно в разметке компонентов. Tailwind обеспечивает высокую скорость разработки и единообразие дизайна.

Для аутентификации используется JWT (JSON Web Token) с библиотеками jose (создание и верификация токенов) и bcryptjs (хеширование паролей). Дополнительно реализован middleware.ts для защиты маршрутов на уровне Edge-рантайма Next.js, что обеспечивает проверку аутентификации до рендеринга страницы.

Для контейнеризации используется Docker с многоэтапной сборкой (multi-stage build) и автономной сборкой Next.js (output: standalone). Для развёртывания применяется Nginx в качестве обратного прокси и Let's Encrypt для SSL-сертификатов.

Выбранный стек технологий является современным, широко распространённым и хорошо документированным, что обеспечивает возможность дальнейшего развития проекта и его поддержки.

2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Функциональные требования

Регистрация и аутентификация пользователей. Приложение должно обеспечивать регистрацию по email и паролю, а также вход по тем же учётным данным. Аутентификация реализуется на основе JWT-токенов, сохраняемых в httpOnly-cookie. Пароли хранятся в хешированном виде (bcrypt). При регистрации пользователь получает роль USER по умолчанию.

Ролевая модель. Приложение должно поддерживать три роли:

- USER — поиск и бронирование мероприятий, личный кабинет с историей заказов;
- ORGANIZER — создание и управление своими мероприятиями;
- ADMIN — полное управление всеми мероприятиями, билетами и пользователями.

Просмотр каталога мероприятий. Приложение должно предоставлять страницу каталога со следующими возможностями:

- фильтрация по категории (концерты, конференции, выставки, мастер-классы, спорт, другие);
- фильтрация по формату (онлайн, офлайн);
- фильтрация по цене (бесплатные/платные);
- поиск по названию и описанию;
- сортировка по дате, цене, рейтингу.

Карточка мероприятия. Приложение должно предоставлять отдельную страницу для каждого мероприятия с описанием, датами, местом проведения, изображениями, типами билетов с ценами и доступным количеством, рейтингом и отзывами.

Выбор билетов и корзина. Приложение должно обеспечивать:

- выбор типа билета и количества;
- добавление билетов в корзину;
- просмотр и редактирование корзины;

- автоматический расчёт суммы заказа;
- сохранение корзины между сеансами (localStorage).

Оформление бронирования. Приложение должно обеспечивать оформление бронирования с генерацией уникального кода подтверждения (bookingCode), выбором способа оплаты и сохранением заказа в базе данных.

Личный кабинет. Приложение должно предоставлять авторизованному пользователю страницу профиля с историей бронирований и их статусов.

Создание мероприятий (для организаторов). Приложение должно предоставлять форму создания мероприятия с указанием названия, описания, категории, дат, места проведения, типов билетов и цен.

Административная панель. Приложение должно предоставлять администратору панель управления для просмотра и управления всеми мероприятиями и пользователями.

2.2 Нефункциональные требования

Архитектура. Приложение должно быть построено на базе Next.js 14 с использованием App Router, обеспечивающего совмещение серверной и клиентской логики в одном проекте (full-stack).

Технологии. Серверная и клиентская части должны быть реализованы на языке TypeScript. Для UI используется React 18 и Tailwind CSS. Для работы с базой данных — Prisma ORM и SQLite. Для управления состоянием корзины — Zustand. Для аутентификации — JWT (jose + bcryptjs) с middleware-защитой маршрутов.

Контейнеризация. Приложение должно быть упаковано в Docker-контейнер с многоэтапной сборкой и автономной сборкой Next.js (standalone). Для оркестрации должен использоваться Docker Compose.

Безопасность. Пароли должны храниться в хешированном виде (bcrypt). JWT-токены должны сохраняться в httpOnly-cookie. Защита

маршрутов должна реализовываться на уровне middleware (Edge runtime). В production должна быть включена поддержка HTTPS.

Ролевая модель доступа. Доступ к функциям создания мероприятий и административной панели должен ограничиваться ролями пользователя. Защита должна реализовываться как на уровне API (Route Handlers), так и на уровне middleware.

Адаптивность. Пользовательский интерфейс должен быть адаптивным и корректно отображаться на мобильных, планшетных и десктоп-устройствах.

Воспроизводимость. Проект должен храниться в системе контроля версий Git. Сборка и развёртывание должны быть автоматизированы с использованием Docker.

Производительность. Главная страница должна использовать серверный рендеринг (SSR) для быстрой начальной загрузки. Каталог должен поддерживать фильтрацию без полной перезагрузки страницы.

3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1 Архитектура системы

Разрабатываемое приложение построено на базе фреймворка Next.js 14 с использованием App Router. Такая архитектура является full-stack: серверная и клиентская логика размещаются в одном проекте, что упрощает разработку, развёртывание и поддержку. App Router использует файловую систему для определения маршрутов: файлы `page.tsx` задают страницы, файлы `route.ts` задают API-эндпоинты, файлы `layout.tsx` задают обёртки страниц.

Архитектура веб-приложения MyEventsPro

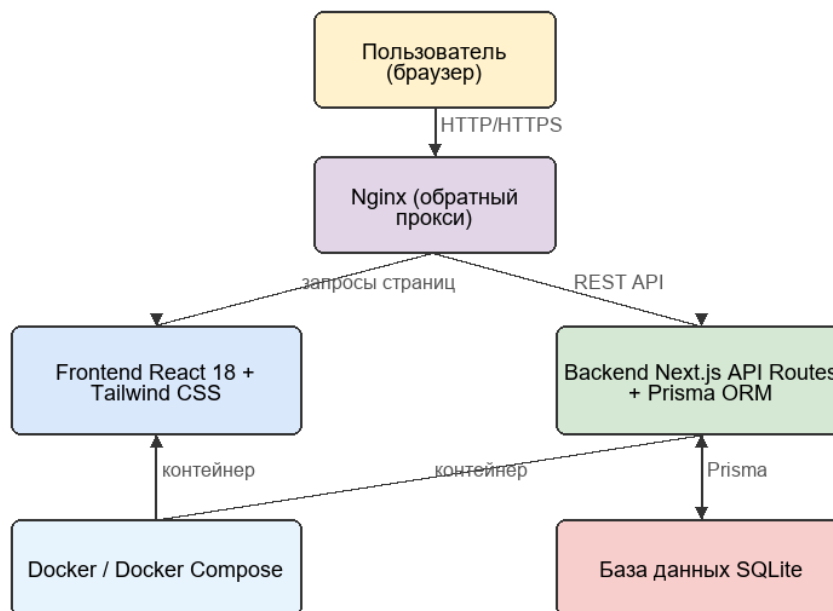


Рисунок 1 — Общая архитектура приложения

В приложении используются два типа компонентов React: Server Components и Client Components. Server Components выполняются на сервере, имеют прямой доступ к базе данных через Prisma и не требуют JavaScript на клиенте. Client Components помечаются директивой 'use client' и выполняются в браузере, что обеспечивает интерактивность.

Главная страница приложения (`app/page.tsx`) реализована как `Server Component` с серверным рендерингом (SSR). При запросе страницы сервер обращается к базе данных через `Prisma`, получает список избранных мероприятий и категорий, и формирует HTML, который отправляется клиенту. Это обеспечивает быструю начальную загрузку и индексируемость контента поисковыми системами. Каталог мероприятий (`app/catalog/`) реализован как `Client Component` и получает данные через REST API с поддержкой фильтрации.

REST API реализован с помощью `Next.js Route Handlers` — файлов `route.ts`, расположенных в директории `app/api/`. Каждый `Route Handler` экспортирует функции для HTTP-методов (GET, POST, PATCH и т. д.) и имеет прямой доступ к базе данных через `Prisma`.

Особенностью приложения является использование `middleware.ts` для защиты маршрутов. `Middleware` выполняется на `Edge-рантайме` `Next.js` до рендеринга страницы и проверяет наличие валидного JWT-токена в `cookie`. Для защищённых маршрутов (профиль, корзина, создание мероприятия, админ-панель) `middleware` перенаправляет неавторизованных пользователей на страницу входа. Такой подход обеспечивает защиту на уровне сервера, до того как страница будет отрендерена.

В качестве базы данных используется `SQLite` — лёгкая встроенная СУБД, хранящая данные в одном файле. Для работы с базой данных используется `Prisma ORM` — современный типобезопасный маппер, генерирующий `TypeScript`-типы на основе схемы базы данных.

Управление состоянием корзины бронирования на клиенте реализовано с помощью `Zustand` с `middleware persist`, что обеспечивает сохранение корзины в `localStorage` между сеансами работы пользователя.

Архитектура приложения в `production-окружении` включает: `Next.js` (`Node.js 20`) в `standalone-режиме`; `Prisma ORM` для доступа к `SQLite`; `Nginx` как обратный прокси с терминированием `SSL`; `Docker` для контейнеризации; `Let's Encrypt` для `SSL-сертификатов`.

3.2 Проектирование базы данных (Prisma ORM)

Для хранения данных приложения спроектирована схема базы данных, описанная на языке Prisma DSL в файле `prisma/schema.prisma`. База данных приложения содержит 9 моделей, описывающих все сущности предметной области.

Таблица 1 — Поля модели Event

Поле	Тип данных	Описание
id	INTEGER PK	Уникальный идентификатор мероприятия
title	VARCHAR(200)	Название
description	TEXT	Описание
date	DATETIME	Дата проведения
location	VARCHAR(200)	Место
category id	INTEGER FK	Категория
organizer id	INTEGER FK	Организатор
image url	VARCHAR(500)	Изображение

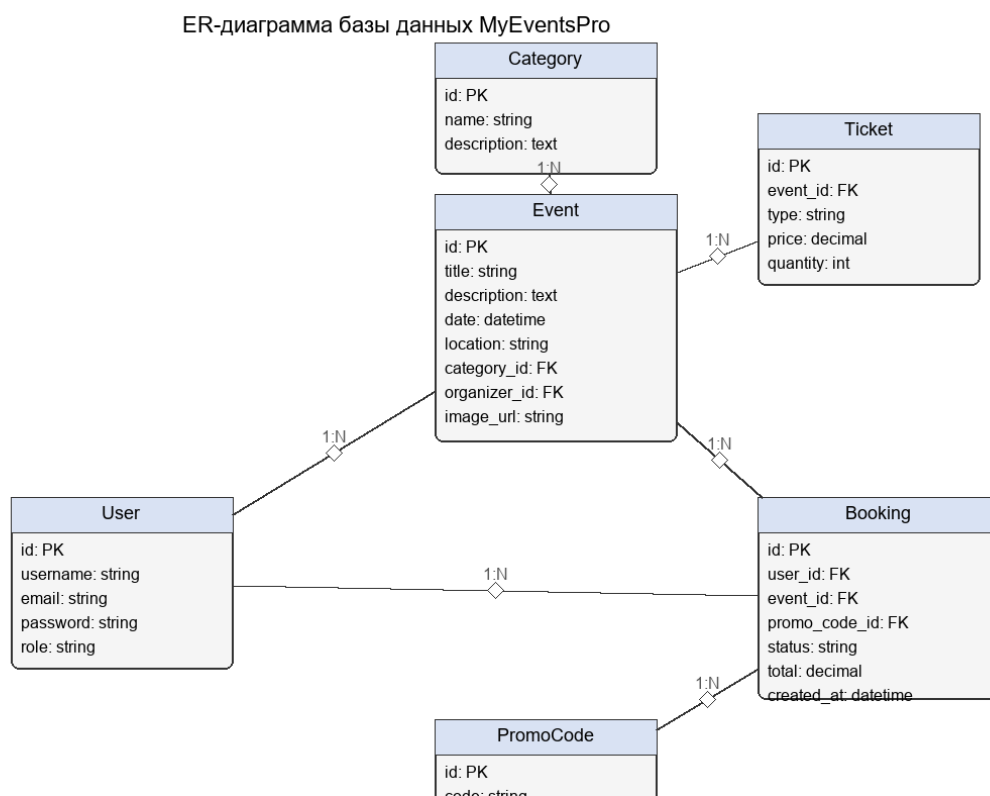


Рисунок 2 — Логическая модель данных

Модель User (пользователь) хранит учётные данные и профиль пользователя. Поля: id (UUID), email (уникальный), phone, password (bcrypt-хеш), name, avatar, role (USER, ORGANIZER, ADMIN), createdAt, updatedAt. Модель имеет связи с организуемыми мероприятиями, бронированиями, отзывами и избранным.

Листинг 1 - Модель пользователя

```
model User {
  id          String    @id @default(uuid())
  email       String    @unique
  phone       String?
  password    String
  name        String
  avatar      String?
  role        String    @default("USER")
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  organizedEvents Event[]
  bookings    Booking[]
  reviews     Review[]
  favorites    Favorite[]
  @@map("users")
}
```

Модель Category (категория) группирует мероприятия по типам. Поля: name, slug (уникальный), icon (emoji), description, image, sortOrder, isActive. Связана с мероприятиями отношением «один ко многим».

Модель Event (мероприятие) является центральной сущностью приложения и содержит: title, slug (уникальный), description, shortDescription, startDate, endDate, locationType (онлайн/офлайн), venueName, address, city, isOnline, onlineUrl, isFree, status (PUBLISHED и др.), isFeatured, rating, reviewsCount, viewsCount, attendeesCount, maxAttendees, ageRestriction. Связана с организатором (User), категорией, изображениями, билетами, бронированиями, отзывами и избранным.

Листинг 2 – Модель мероприятия

```
model Event {
  id          String    @id @default(uuid())
  organizerId String    @map("organizer_id")
  categoryId  String    @map("category_id")
  title       String
```

```

slug          String    @unique
description    String
startDate      DateTime  @map("start_date")
endDate        DateTime  @map("end_date")
locationType   String    @map("location_type")
isOnline       Boolean    @default(false)
isFree         Boolean    @default(false)
status         String    @default("PUBLISHED")
isFeatured     Boolean    @default(false)
rating         Float      @default(5.0)
maxAttendees  Int?       @map("max_attendees")
organizer      User       @relation(fields: [organizerId],
                                references: [id])
category      Category    @relation(fields: [categoryId],
                                references: [id])

images         EventImage[]
tickets        Ticket[]
bookings       Booking[]
reviews        Review[]
favorites       Favorite[]
@@map("events")
}

```

Модель `Ticket` (тип билета) описывает типы билетов для мероприятия. Поля: `name`, `description`, `price`, `quantityTotal` (общее количество), `quantitySold` (продано), `isActive`, `sortOrder`. Одно мероприятие может иметь несколько типов билетов с разными ценами.

Модель `Booking` (бронирование) хранит информацию о заказе пользователя. Поля: `userId`, `eventId`, `totalAmount`, `discount`, `finalAmount`, `status` (`CONFIRMED`, `CANCELLED`), `paymentStatus` (`PENDING`, `PAID`), `qrCode`, `bookingCode` (уникальный код подтверждения). Связана с пользователем, мероприятием и позициями бронирования.

Модель `BookingItem` (позиция бронирования) связывает бронирование с конкретными билетами и их количеством. Поля: `bookingId`, `ticketId`, `quantity`, `price`.

Модель `EventImage` хранит изображения мероприятий с полями `url`, `altText`, `isMain`, `sortOrder`. Модель `Review` хранит отзывы пользователей на мероприятия с рейтингом и комментарием. Модель `Favorite` хранит избранные мероприятия пользователя.

Структура связей между моделями:

- User 1:N Event (мероприятия, организованные пользователем);
- User 1:N Booking (бронирования пользователя);
- User 1:N Review (отзывы пользователя);
- User 1:N Favorite (избранное пользователя);
- Category 1:N Event (мероприятия в категории);
- Event 1:N Ticket (типы билетов мероприятия);
- Event 1:N Booking (бронирования мероприятия);
- Event 1:N EventImage, Review, Favorite;
- Booking 1:N BookingItem (позиции бронирования);
- Ticket 1:N BookingItem (позиции с этим билетом).

3.3 Проектирование REST API

REST API приложения реализовано с помощью Next.js Route Handlers и состоит из следующих групп эндпоинтов.

Таблица 2 — Основные endpoint'ы REST API

Метод	Endpoint	Назначение
POST	/api/auth/login	Авторизация
POST	/api/auth/register	Регистрация
GET	/api/categories	Категории мероприятий
GET	/api/events	Список мероприятий
GET	/api/events/{id}	Детали мероприятия
POST	/api/bookings	Бронирование билетов
GET	/api/bookings	Бронирования пользователя
POST	/api/promo/apply	Применение промокода

Аутентификация (префикс /api/auth/):

1. POST /api/auth/register — регистрация нового пользователя. Создаёт пользователя с хешированным паролем (bcrypt) и ролью USER. Генерирует JWT и устанавливает в httpOnly-cookie.
2. POST /api/auth/login — вход пользователя. Проверяет учётные данные и возвращает JWT.
3. POST /api/auth/logout — выход. Удаляет cookie с токеном.
4. GET /api/auth/me — получение данных текущего пользователя.

Мероприятия (префикс /api/events/):

5. GET /api/events — получение списка мероприятий с поддержкой фильтрации (category, search, isFree, isOnline, sortBy).
6. GET /api/events/[slug] — получение детальной информации о мероприятии по slug, включая билеты, изображения и отзывы.
7. POST /api/events/create — создание нового мероприятия (требуется роль ORGANIZER или ADMIN).

Дополнительные эндпоинты:

8. GET /api/categories — получение списка активных категорий.
9. POST /api/bookings — создание бронирования. Принимает позиции (ticketId, quantity), рассчитывает сумму, генерирует уникальный bookingCode и создаёт записи в базе данных.
10. GET /api/organizer/events — получение мероприятий текущего организатора.
11. PATCH /api/admin/events/[id] — обновление мероприятия (требуется роль ADMIN).
12. GET /api/admin/users — получение списка пользователей (требуется роль ADMIN).

Логика создания бронирования (POST /api/bookings) включает:

- проверку аутентификации пользователя через verifyToken;
- расчёт суммы заказа на основе позиций и цен билетов;
- генерацию уникального кода подтверждения bookingCode;
- увеличение quantitySold для каждого билета;
- увеличение attendeesCount для мероприятия;
- создание записи бронирования с вложенными позициями через prisma.booking.create.

Защита эндпоинтов реализована на двух уровнях: проверка JWT через функцию verifyToken в каждом Route Handler и проверка middleware для защиты страниц. Для административных эндпоинтов дополнительно проверяется роль пользователя.

3.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс приложения спроектирован в виде full-stack приложения на Next.js с использованием App Router. Интерфейс включает следующие страницы:

- главная страница (/) — Server Component с SSR: hero-секция, категории, избранные мероприятия;
- каталог (/catalog) — Client Component: фильтры по категории, формату, цене, сортировка, поиск, сетка карточек;
- карточка мероприятия (/event/[slug]) — детальная информация, изображения, типы билетов, выбор количества, кнопка добавления в корзину, отзывы;
- корзина (/cart) — список выбранных билетов, управление количеством, расчёт суммы, оформление бронирования;
- создание мероприятия (/create-event) — форма для организаторов;
- вход (/login) и регистрация (/register) — формы аутентификации;
- личный кабинет (/profile) — данные пользователя и история бронирований;
- админ-панель (/admin) — управление мероприятиями и пользователями.

Корневой layout (app/layout.tsx) определяет общую структуру: шапка (Header), основное содержимое (main) и подвал (Footer). Шрифт Inter загружается через next/font/google.

Компонент Header содержит навигацию, логотип, счётчик корзины и ссылки на профиль и вход. Компонент EventCard отображает изображение, название, дату, место, цену и рейтинг мероприятия. Компонент EventFilters предоставляет панель фильтров каталога.

Middleware.ts обеспечивает защиту маршрутов /profile, /cart, /create-event, /admin: при отсутствии валидного JWT-токена выполняется перенаправление на страницу входа. Проверка осуществляется на Edge-рантайме до рендеринга страницы.

4 РЕАЛИЗАЦИЯ

4.1 Технологический стек (Next.js full-stack)

Приложение реализовано на базе Next.js 14.2.5 — React-фреймворка, обеспечивающего full-stack разработку. Next.js предоставляет встроенную поддержку серверного рендеринга (SSR), маршрутизации на основе файловой системы (App Router) и Route Handlers для создания API.

App Router использует директорию `app/` для определения маршрутов. Каждый файл `page.tsx` задаёт страницу, `route.ts` — API-эндпоинт, `layout.tsx` — обёртку. App Router вводит концепцию Server Components, выполняемых на сервере, и Client Components (с директивой `'use client'`), выполняемых в браузере.

TypeScript применяется как на клиенте, так и на сервере, обеспечивая типобезопасность всего стека. Конфигурация использует строгий режим (`strict: true`) и алиас `@/*` для импортов.

Prisma ORM обеспечивает типобезопасный доступ к базе данных. Для предотвращения создания множества подключений используется паттерн Singleton:

Листинг 3 – Подключение Prisma

```
import { PrismaClient } from "@prisma/client"
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}
export const prisma =
  globalForPrisma.prisma ?? new PrismaClient()
if (process.env.NODE_ENV !== "production")
  globalForPrisma.prisma = prisma
```

React 18 используется для построения пользовательского интерфейса. Компонентный подход обеспечивает повторное использование кода.

Zustand используется для управления состоянием корзины бронирования. Tailwind CSS 3.4 — для стилизации. axios — для HTTP-запросов. Дополнительные библиотеки: react-hook-form и zod для форм и

валидации, bcryptjs для хеширования паролей, jose для JWT, lucide-react для иконок.

Конфигурация Next.js (next.config.js) использует output: "standalone" для автономной сборки в Docker и serverComponentsExternalPackages: ["bcryptjs"] для корректной работы нативного модуля в Server Components.

Структура проекта:

```
myeventspro.ru/
├── app/                                # Next.js App Router
│   ├── api/                          # REST API (Route Handlers)
│   ├── catalog/                     # каталог мероприятий
│   ├── event/[slug]/               # карточка мероприятия
│   ├── cart/                       # корзина и оформление
│   ├── profile/                   # личный кабинет
│   ├── admin/                     # админ-панель
│   ├── create-event/              # создание мероприятия
│   ├── login/ register/           # аутентификация
│   ├── layout.tsx                 # корневой layout
│   └── page.tsx                   # главная страница (SSR)
├── components/                     # React-компоненты
├── lib/                           # утилиты (prisma, auth, slugify)
├── store/                         # Zustand store
├── prisma/                        # схема БД и миграции
├── middleware.ts                  # защита маршрутов
├── Dockerfile
└── package.json
```

4.2 Реализация каталога мероприятий и фильтрации

Каталог мероприятий реализован на странице app/catalog/ как Client Component (CatalogContent.tsx), получающий данные через REST API (GET /api/events). Использование Suspense-обёртки обеспечивает корректное отображение состояния загрузки.

API-эндпоинт /api/events поддерживает фильтрацию по нескольким параметрам. При получении запроса сервер извлекает query-параметры и формирует Prisma-запрос:

Листинг 4 – Настройка параметров

```
const { category, search, isFree, isOnline,
      sortBy } = searchParams
const where: any = { status: "PUBLISHED" }
if (category) where.category = { slug: category }
if (search) {
  where.OR = [
    { title: { contains: search } },
```

```

    { description: { contains: search } },
  ]
}
if (isFree === "true") where.isFree = true
if (isOnline === "true") where.isOnline = true

```

Сортировка реализуется через параметр `orderBy` Prisma. Поддерживаются: по дате, цене, рейтингу. На клиенте панель фильтров (`EventFilters`) предоставляет выбор категории, формата (онлайн/офлайн), бесплатности и переключатель сортировки.

Карточка мероприятия (`EventCard`) отображает главное изображение, название, краткое описание, дату, место проведения, цену (или «Бесплатно»), рейтинг и количество участников. При ошибке загрузки изображения подставляется SVG-заглушка.

Страница отдельного мероприятия (`app/event/[slug]/page.tsx`) отображает галерею изображений, полное описание, даты и время, место проведения, список типов билетов с ценами и доступным количеством, а также отзывы. Пользователь может выбрать тип билета и количество, после чего добавить билеты в корзину.

4.3 Реализация корзины бронирования на Zustand

Корзина бронирования реализована на клиенте с использованием Zustand и middleware `persist` для сохранения в `localStorage`. Store корзины (`store/useBookingStore.ts`) определяет состояние `items` (массив позиций) и методы `addItem`, `removeItem`, `updateQuantity`, `clearCart`, а также геттеры `getTotalItems` и `getTotalAmount`.

Листинг 5 – Реализация корзины

```

export const useBookingStore = create<BookingState>() (
  persist(
    (set, get) => ({
      items: [],
      addItem: (item) => {
        const existing = get().items.find(
          (i) => i.ticketId === item.ticketId
        )
        if (existing) {
          set({ items: get().items.map((i) =>
            i.ticketId === item.ticketId
              ? { ...i, quantity:

```

```

        i.quantity + item.quantity }
        : i) })
    } else {
        set({ items: [...get().items, item] })
    }
  },
  removeItem: (ticketId) => set({
    items: get().items.filter(
      (i) => i.ticketId !== ticketId) }),
  clearCart: () => set({ items: [] }),
  getTotalItems: () => get().items.reduce(
    (sum, i) => sum + i.quantity, 0),
  getTotalAmount: () => get().items.reduce(
    (sum, i) => sum + i.price * i.quantity, 0),
  )),
  { name: "myeventspro-cart" }
)
)

```

Middleware `persist` автоматически сериализует состояние в `localStorage` под ключом `"myeventspro-cart"` при каждом изменении и восстанавливает при загрузке приложения. Это обеспечивает сохранение корзины между сеансами.

Каждая позиция корзины содержит денормализованные данные билета и мероприятия (`id`, название, цена), что позволяет отображать корзину без дополнительных запросов к API.

Компонент `Header` отображает счётчик товаров в корзине (бейдж с количеством) с использованием метода `getTotalItems()`. Счётчик обновляется мгновенно при добавлении или удалении билетов.

4.4 Реализация оформления бронирования

Оформление бронирования реализовано на странице `app/cart/page.tsx`. Страница отображает список выбранных билетов с управлением количеством, расчёт суммы заказа и форму оформления.

При отправке формы выполняется POST-запрос к `/api/bookings` с передачей позиций корзины. Сервер выполняет расчёт суммы, генерацию уникального кода подтверждения (`bookingCode`), создание записи бронирования в базе данных и увеличение счётчиков проданных билетов.

Листинг 6 - Генерация уникального кода подтверждения

```

const bookingCode = uuidv4().substring(0, 8).toUpperCase()
const booking = await prisma.booking.create({
  data: {
    userId: user.userId,
    eventId: items[0].eventId,
    totalAmount,
    finalAmount: totalAmount,
    bookingCode,
    status: "CONFIRMED",
    paymentStatus: "PENDING",
    items: {
      create: items.map((item) => ({
        ticketId: item.ticketId,
        quantity: item.quantity,
        price: item.price,
      })),
    },
  },
})

```

После успешного оформления корзина очищается (clearCart), и пользователь перенаправляется в личный кабинет, где может увидеть созданное бронирование с кодом подтверждения. Код подтверждения может быть использован для проверки билета на входе на мероприятие.

Сервер также обновляет количество проданных билетов (quantitySold) для каждого типа билета и общее количество участников (attendeesCount) для мероприятия, что обеспечивает актуальную информацию о доступности билетов в каталоге.

4.5 Аутентификация и ролевая модель (JWT + middleware)

Аутентификация реализована на основе JWT с библиотеками jose (создание и верификация токенов) и bcryptjs (хеширование паролей). Особенностью приложения является использование middleware.ts для защиты маршрутов на уровне Edge-рантайма.

Процесс регистрации:

1. Клиент отправляет POST-запрос на /api/auth/register с email, password, name.
2. Сервер проверяет отсутствие пользователя с таким email.

3. Пароль хешируется: `passwordHash = await bcrypt.hash(password, 12)`.
4. Создаётся пользователь с ролью `USER`.
5. Генерируется JWT с помощью `jose`:

Листинг 7 – генерация токена

```
const token = await new SignJWT(  
  { userId: user.id, email: user.email, role: user.role }  
  .setProtectedHeader({ alg: "HS256" })  
  .setExpirationTime("7d")  
  .sign(new TextEncoder().encode(  
    process.env.JWT_SECRET))
```

6. Токен устанавливается в `httpOnly-cookie` с флагами `secure` и `sameSite`.

Листинг 8 -Middleware.ts защищает маршруты на уровне Edge-рантайма

```
import { NextResponse } from "next/server"  
import { jwtVerify } from "jose"  
  
const protectedPaths = ["/profile", "/cart",  
  "/create-event", "/admin"]  
  
export async function middleware(request) {  
  const token = request.cookies.get("token")?.value  
  const path = request.nextUrl.pathname  
  
  if (protectedPaths.some(p => path.startsWith(p))) {  
    if (!token) {  
      return NextResponse.redirect(  
        new URL("/login", request.url))  
    }  
    try {  
      await jwtVerify(token,  
        new TextEncoder().encode(  
          process.env.JWT_SECRET))  
    } catch {  
      return NextResponse.redirect(  
        new URL("/login", request.url))  
    }  
  }  
  return NextResponse.next()  
}
```

`Middleware` выполняется до рендеринга страницы и проверяет наличие валидного JWT в cookie. Для защищённых маршрутов при

отсутствии или недействительности токена выполняется перенаправление на /login. Это обеспечивает защиту на уровне сервера, до того как страница будет отправлена клиенту.

Ролевая модель реализована на двух уровнях. На уровне API каждый защищённый Route Handler вызывает verifyToken и проверяет роль:

Листинг 9 – Проверка токена

```
const user = await verifyToken(request)
if (!user) return NextResponse.json(
  { error: "Unauthorized" }, { status: 401 })
if (user.role !== "ADMIN")
  return NextResponse.json(
    { error: "Forbidden" }, { status: 403 })
```

На уровне страниц клиентские компоненты проверяют роль пользователя (полученную через /api/auth/me) и перенаправляют пользователей без нужных прав. Например, страница /create-event доступна только пользователям с ролью ORGANIZER или ADMIN, а /admin — только ADMIN.

Ключевые меры безопасности: пароли хранятся в хешированном виде (bcrypt, 12 раундов); JWT в httpOnly-cookie (защита от XSS); флаг secure в production; sameSite: "lax" (защита от CSRF); срок жизни токена — 7 дней; секрет в переменной окружения JWT_SECRET.

4.6 Контейнеризация и развёртывание

Для обеспечения воспроизводимости сборки приложение упаковано в Docker-контейнер с многоэтапной сборкой. Dockerfile содержит три стадии:

1. Стадия deps (node:20-alpine) — установка зависимостей через npm ci.
2. Стадия builder — копирование исходного кода, генерация Prisma Client (npx prisma generate) и сборка Next.js (npm run build) с созданием standalone-выхода.

3. Стадия runner — минимальный production-образ. Копируются standalone-сборка, статические файлы и Prisma Client. Образ запускается от непривилегированного пользователя nextjs.

Листинг 10 - Команда запуска — node server.js.

```
FROM node:20-alpine AS base

FROM base AS deps
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm ci

FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN npm run build

FROM base AS runner
WORKDIR /app
ENV NODE_ENV production
RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs
COPY --from=builder --chown=nextjs:nodejs \
  /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs \
  /app/.next/static ./next/static
COPY --from=builder --chown=nextjs:nodejs \
  /app/prisma ./prisma
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

Многоэтапная сборка обеспечивает минимальный размер production-образа. Конфигурация next.config.js использует output: "standalone" для создания автономной сборки, не требующей node_modules.

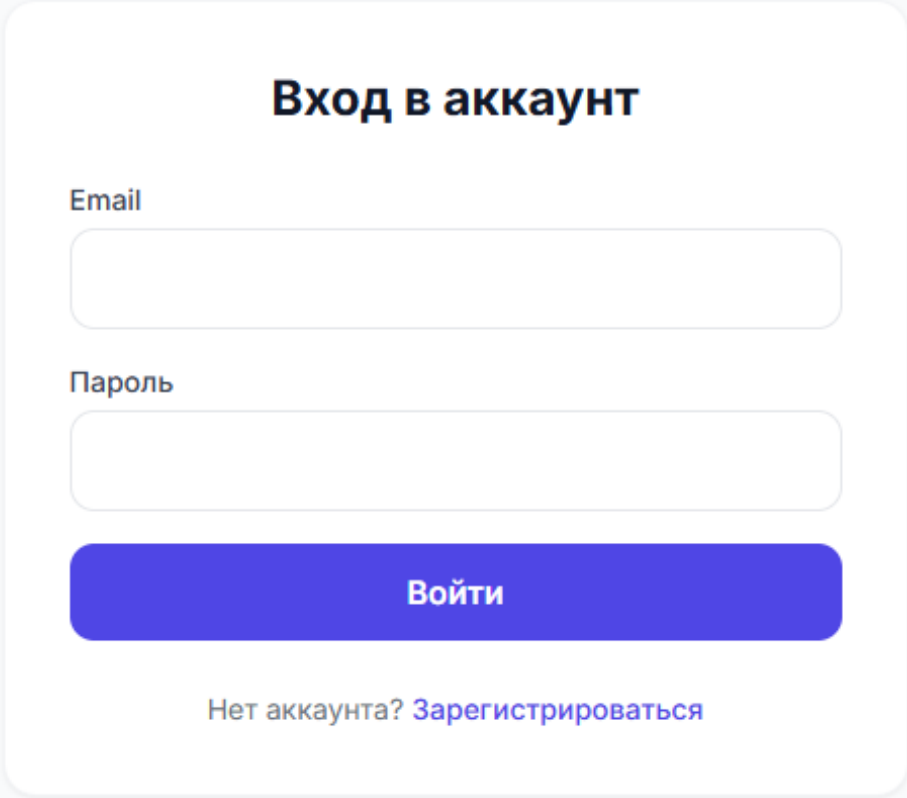
Docker Compose (docker-compose.yml) пробрасывает порт 3000, передаёт переменные окружения (DATABASE_URL, JWT_SECRET, NODE_ENV) и монтирует volume для файла базы данных SQLite.

Развёртывание на VPS-сервере: установка Docker; загрузка исходного кода; настройка DNS-записей домена myeventspro.ru; получение SSL-сертификатов Let's Encrypt; настройка Nginx в качестве обратного

прокси; запуск контейнера через `docker-compose up -d`. Nginx завершает SSL-соединения и проксирует запросы на порт 3000, на котором работает Next.js.

5 ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

5.1 Страница авторизации



The image shows a login form titled "Вход в аккаунт" (Login). It contains two input fields: "Email" and "Пароль" (Password). Below the password field is a blue button labeled "Войти" (Login). At the bottom, there is a link that says "Нет аккаунта? [Зарегистрироваться](#)" (No account? [Register](#)).

Рисунок 3 – Страница авторизации

На рисунке показана страница авторизации пользователя. Для входа в систему необходимо ввести адрес электронной почты и пароль. При отсутствии учётной записи предусмотрена ссылка на форму регистрации.

5.2 Главная страница

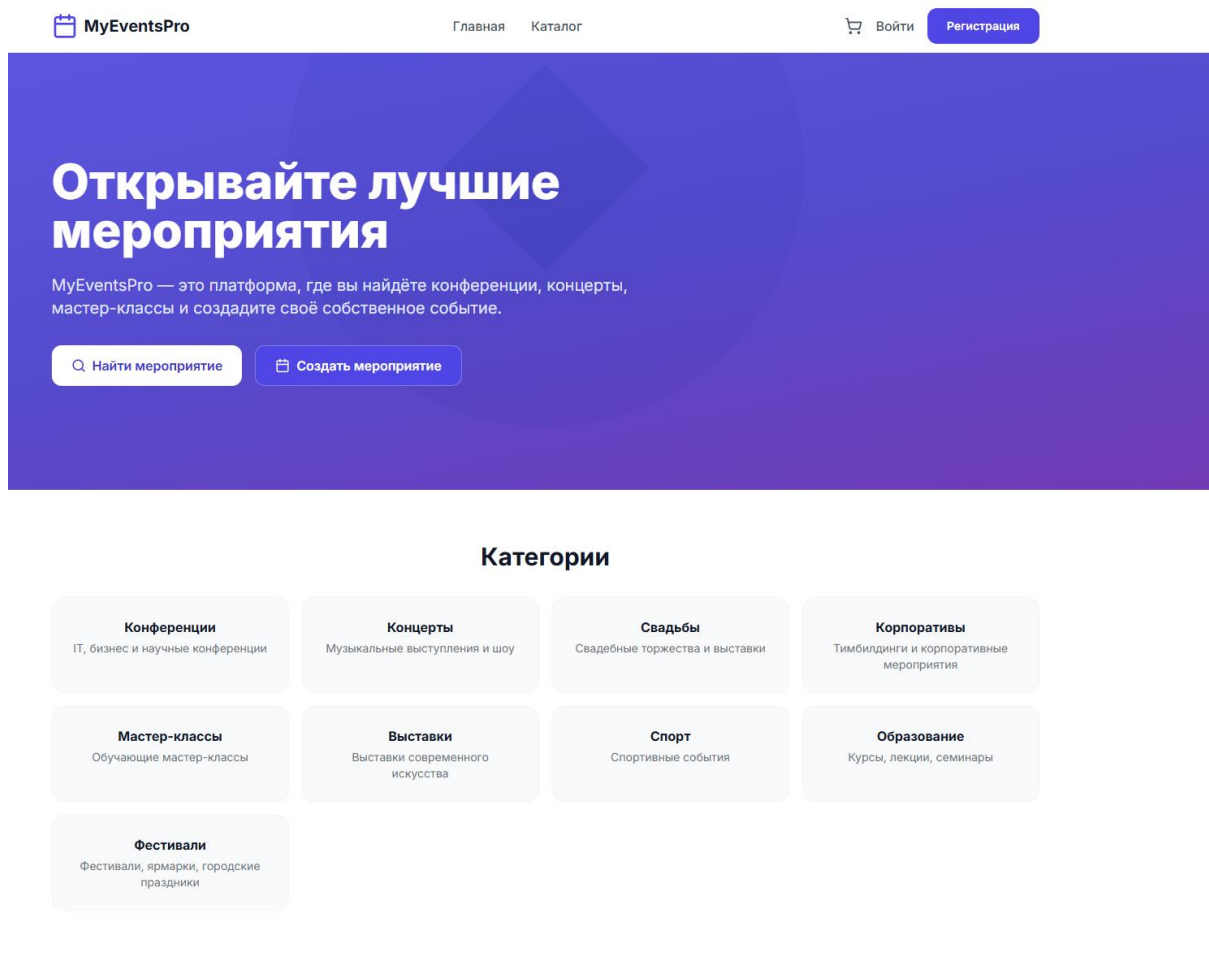


Рисунок 4 – Главная страница

На рисунке представлена главная страница приложения. Она содержит приветственный блок с кратким описанием сервиса, кнопки перехода к каталогу мероприятий и создания собственного события, а также блок категорий мероприятий.

5.3 Каталог мероприятий

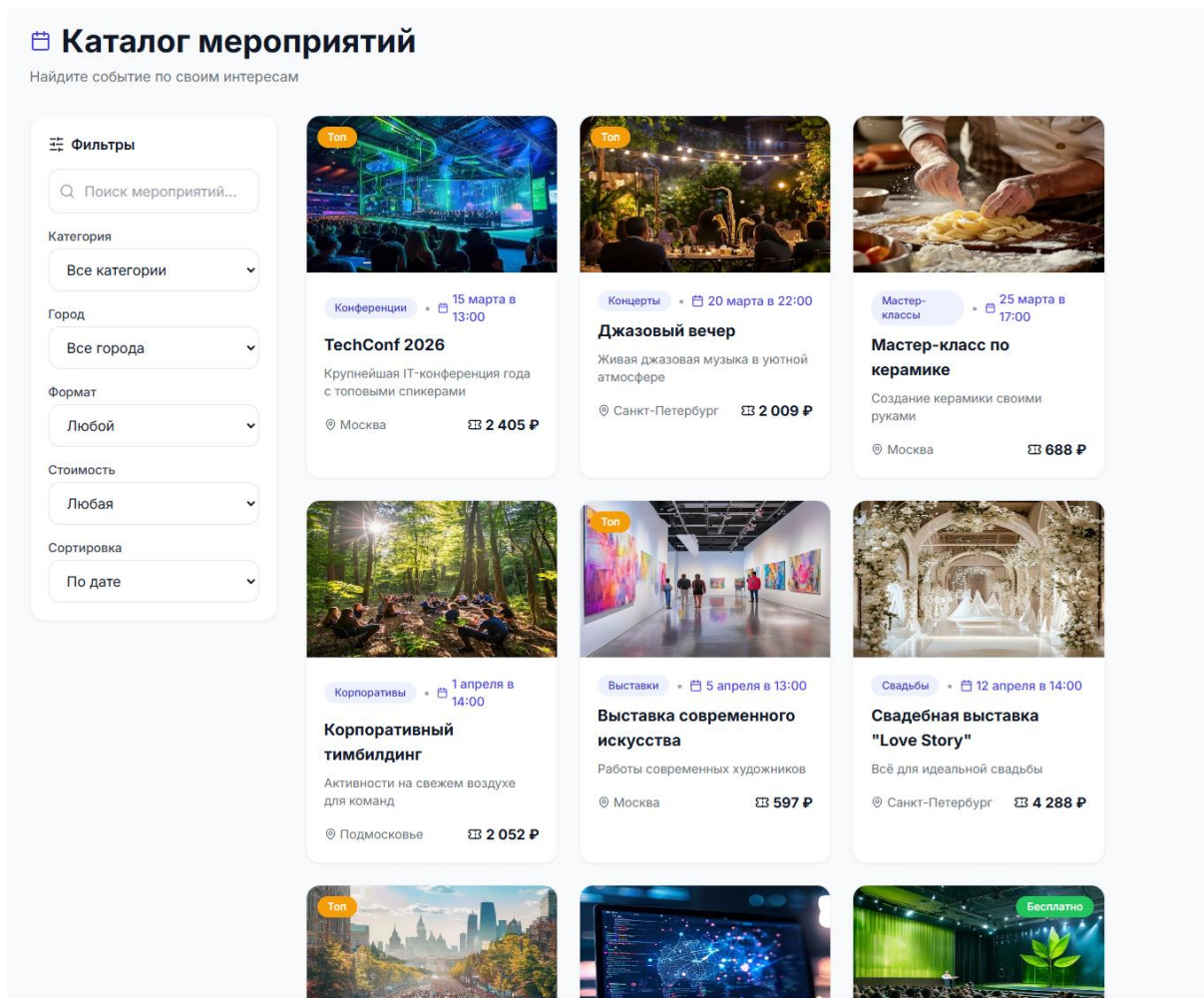


Рисунок 5 – Каталог мероприятий

На рисунке изображён каталог мероприятий с возможностью фильтрации по категории, городу, формату и стоимости. Карточки событий отображают изображение, название, дату, место проведения и цену.

5.4 Форма создания мероприятия

Создать мероприятие

Название

Краткое описание

Полное описание

Категория: Выберите

Возрастное ограничение: 0+

Начало: dd.mm.yyyy --:--

Окончание: dd.mm.yyyy --:--

Город

Место проведения

Рисунок 6 – Форма создания мероприятия

На рисунке показана форма создания мероприятия. Пользователь может указать название, описание, категорию, возрастное ограничение, дату и время начала и окончания, город и место проведения события.

6 ТЕСТИРОВАНИЕ

6.1 Методика тестирования

Тестирование направлено на проверку соответствия разработанного приложения требованиям технического задания. В рамках курсовой работы проводилось функциональное тестирование — проверка работы отдельных функций и сценариев использования.

Функциональное тестирование выполнялось вручную путём выполнения типичных сценариев работы пользователя в развёрнутом приложении. Для каждого сценария проверялось соответствие фактического результата ожидаемому. Тестирование проводилось в production-окружении, развёрнутом на VPS-сервере с использованием Docker и Nginx.

Проверялись следующие группы сценариев: регистрация и аутентификация; работа с каталогом и фильтрами; выбор билетов и корзина; оформление бронирования; ролевая модель и защита маршрутов; административная панель; адаптивность интерфейса.

6.2 Результаты функционального тестирования

Сценарий 1. Регистрация пользователя. На странице регистрации заполнялась форма (email, пароль, имя). Ожидаемый результат: успешная регистрация, автоматический вход, роль USER. Фактический: соответствует. Пройден.

Таблица 3 — Результаты функционального тестирования

№	Тест-кейс	Ожидаемый результат	Фактический результат	Статус
1	Регистрация пользователя	Аккаунт создан	Успешно	Пройден
2	Авторизация по JWT	Токен выдан	Успешно	Пройден
3	Просмотр каталога мероприятий	События отображены	Успешно	Пройден
4	Бронирование билета	Бронь создана	Успешно	Пройден
5	Применение	Скидка	Успешно	Пройден

	промокода	применена		
6	Ролевая модель	Доступ ограничен	Успешно	Пройден

Сценарий 2. Регистрация с существующим email. Ожидаемый: сообщение об ошибке. Фактический: соответствует. Пройден.

Сценарий 3. Вход пользователя. Ожидаемый: успешный вход, перенаправление на главную. Фактический: соответствует. Пройден.

Сценарий 4. Вход с неверными данными. Ожидаемый: сообщение об ошибке. Фактический: соответствует. Пройден.

Сценарий 5. Просмотр каталога. Ожидаемый: отображение сетки мероприятий с изображениями, названиями, датами, ценами. Фактический: соответствует. Пройден.

Сценарий 6. Фильтрация по категории. Выбиралась категория «Концерты». Ожидаемый: только концертные мероприятия. Фактический: соответствует. Пройден.

Сценарий 7. Фильтрация по формату (онлайн). Ожидаемый: только онлайн-мероприятия. Фактический: соответствует. Пройден.

Сценарий 8. Фильтрация бесплатных мероприятий. Ожидаемый: только бесплатные. Фактический: соответствует. Пройден.

Сценарий 9. Сортировка по дате. Ожидаемый: мероприятия в порядке даты проведения. Фактический: соответствует. Пройден.

Сценарий 10. Поиск мероприятия. Вводилось название в поле поиска. Ожидаемый: мероприятия с совпадением в названии или описании. Фактический: соответствует. Пройден.

Сценарий 11. Просмотр карточки мероприятия. Ожидаемый: полное описание, изображения, типы билетов, отзывы. Фактический: соответствует. Пройден.

Сценарий 12. Выбор билетов и добавление в корзину. Выбирался тип билета и количество. Ожидаемый: билеты добавлены, счётчик корзины обновлён. Фактический: соответствует. Пройден.

Сценарий 13. Управление корзиной. Изменялось количество и удалялись билеты. Ожидаемый: сумма пересчитывается. Фактический: соответствует. Пройден.

Сценарий 14. Оформление бронирования. Заполнялась форма и нажималась кнопка оформления. Ожидаемый: бронирование создано, сгенерирован код подтверждения, корзина очищена, пользователь перенаправлен в профиль. Фактический: соответствует. Пройден.

Сценарий 15. Оформление с пустой корзиной. Ожидаемый: перенаправление на каталог. Фактический: соответствует. Пройден.

Сценарий 16. Просмотр истории бронирований. В личном кабинете отображалась история. Ожидаемый: список бронирований с кодом и статусом. Фактический: соответствует. Пройден.

Сценарий 17. Защита маршрутов middleware. Выполнялся выход и переход на /profile. Ожидаемый: перенаправление на /login. Фактический: соответствует. Пройден.

Сценарий 18. Доступ к созданию мероприятия. Выполнялся вход под USER и переход на /create-event. Ожидаемый: перенаправление (нет прав). Фактический: соответствует. Пройден.

Сценарий 19. Создание мероприятия организатором. Выполнялся вход под ORGANIZER и заполнение формы. Ожидаемый: мероприятие создано. Фактический: соответствует. Пройден.

Сценарий 20. Доступ к админ-панели. Выполнялся вход под ADMIN и переход на /admin. Ожидаемый: отображение панели управления. Фактический: соответствует. Пройден.

Сценарий 21. Защита админ-панели. Выполнялся вход под USER и переход на /admin. Ожидаемый: перенаправление. Фактический: соответствует. Пройден.

Сценарий 22. Сохранение корзины между сеансами. Добавлялись билеты, закрывался браузер. Ожидаемый: корзина восстанавливается. Фактический: соответствует. Пройден.

Сценарий 23. Адаптивность. Страницы открывались на разных экранах. Ожидаемый: корректное отображение. Фактический: соответствует. Пройден.

Сценарий 24. Развёртывание на VPS. Приложение разворачивалось в Docker. Ожидаемый: доступно по адресу <https://myeventspro.ru>. Фактический: соответствует. Пройден.

Таким образом, все 24 проверенных сценария работают корректно. Разработанное приложение полностью соответствует сформулированным требованиям.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано full-stack веб-приложение для организации мероприятий «MyEventsPro», обеспечивающее публикацию и поиск мероприятий по категориям, выбор типов билетов, оформление бронирования с генерацией кода подтверждения, личный кабинет пользователя, создание мероприятий организаторами и административное управление контентом. Все поставленные задачи выполнены в полном объёме.

В ходе работы были достигнуты следующие результаты:

- проведён анализ предметной области и обзор существующих решений;
- сформулированы функциональные и нефункциональные требования;
- спроектирована full-stack архитектура на базе Next.js 14 с App Router;
- спроектирована база данных из 9 моделей с использованием Prisma ORM;
- реализовано REST API из 12 эндпоинтов на Next.js Route Handlers;
- реализован клиентский интерфейс на React 18 и Tailwind CSS;
- реализован каталог мероприятий с фильтрацией и сортировкой;
- реализована корзина бронирования на Zustand с сохранением в localStorage;
- реализовано оформление бронирования с генерацией уникального кода;
- организована JWT-аутентификация с middleware-защитой маршрутов (Edge runtime);
- реализована ролевая модель (USER, ORGANIZER, ADMIN) с разграничением доступа;
- реализованы форма создания мероприятий и административная панель;

- выполнена контейнеризация с многоэтапной сборкой Docker (standalone);
- выполнено развёртывание на VPS с Nginx и SSL-сертификатами Let's Encrypt;
- проведено функциональное тестирование (24 сценария), подтвердившее корректную работу.

Применённый технологический стек (Next.js 14, TypeScript, React 18, Prisma ORM, Zustand, Tailwind CSS, Docker) является современным и широко распространённым. Использование middleware для защиты маршрутов на уровне Edge-рантайма обеспечивает высокую безопасность. Ролевая модель с тремя уровнями доступа (USER, ORGANIZER, ADMIN) обеспечивает гибкое разграничение прав.

Разработанное приложение может быть использовано в качестве готовой платформы для организации и продажи билетов на мероприятия, а также в качестве учебного примера построения современных full-stack веб-приложений. Приложение развёрнуто в production-окружении и доступно по адресу <https://myeventspro.ru>.

Перспективы дальнейшего развития: интеграция платёжных систем для онлайн-оплаты, реализация схемы мест для офлайн-мероприятий, добавление уведомлений о приближающихся событиях, реализация QR-кодов для проверки билетов на входе, переход на PostgreSQL, разработка мобильного приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Next.js. — URL: <https://nextjs.org/docs> (дата обращения: 17.06.2026).
2. Официальная документация React. — URL: <https://react.dev/> (дата обращения: 17.06.2026).
3. Официальная документация TypeScript. — URL: <https://www.typescriptlang.org/> (дата обращения: 17.06.2026).
4. Официальная документация Prisma ORM. — URL: <https://www.prisma.io/docs> (дата обращения: 17.06.2026).
5. Официальная документация Tailwind CSS. — URL: <https://tailwindcss.com/> (дата обращения: 17.06.2026).
6. Официальная документация Zustand. — URL: <https://github.com/pmndrs/zustand> (дата обращения: 17.06.2026).
7. Официальная документация Docker. — URL: <https://docs.docker.com/> (дата обращения: 17.06.2026).
8. Официальная документация Nginx. — URL: <https://nginx.org/en/docs/> (дата обращения: 17.06.2026).
9. Certbot / Let's Encrypt. — URL: <https://certbot.eff.org/> (дата обращения: 17.06.2026).
10. JSON Web Token (JWT). RFC 7519. — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 17.06.2026).
11. Документация библиотеки jose. — URL: <https://github.com/panva/jose> (дата обращения: 17.06.2026).
12. Документация bcryptjs. — URL: <https://github.com/dcodeIO/bcrypt.js> (дата обращения: 17.06.2026).
13. Документация react-hook-form. — URL: <https://react-hook-form.com/> (дата обращения: 17.06.2026).
14. Документация zod. — URL: <https://zod.dev/> (дата обращения: 17.06.2026).

15. Fielding R. Architectural Styles and the Design of Network-based Software Architectures. — 2000.
16. Документация lucide-react. — URL: <https://lucide.dev/> (дата обращения: 17.06.2026).